MINS

Mathematics, Implementation, API Adaptation, and Termination Rules

Implementation guide for `mins` version `0.1.0.dev3`

Code audit: repository commit `934195f`

Abstract

This document explains the mathematics implemented by the current Morph-assisted nested importance sampling (MINS) code and connects each equation to the corresponding stage of the Python execution path. It also specifies the public input contracts, shows which arguments must change for common alternative user setups, describes the returned weighted posterior, and gives the exact scientific, resource-limit, and exception termination semantics. The description is an implementation audit of the repository version stated above; it is not a generic account of all algorithms that might be called MINS.

Contents

1 What the current code does

MINS is a *post-processing evidence estimator*. The user first provides posterior samples obtained by some other method. MorphZ fits a fixed normalized density $q(\theta)$ to those samples. The sampler then rewrites the Bayesian evidence integral as a nested-sampling problem under q, performs independent constrained rejection draws from that fixed proposal, and constructs evidence and posterior weights using a deterministic prior-volume trajectory.

At a high level, one run performs the following operations:

1. validate the model, proposal, run limits, and stopping configuration;
2. draw N new live points from the fixed proposal q;
3. evaluate the likelihood, normalized prior, proposal density, and transformed pseudo-likelihood at every point;
4. repeatedly discard the live point with the smallest transformed pseudo-likelihood and replace it with an independent draw satisfying the new constraint;
5. add the discarded point's deterministic quadrature contribution;
6. calculate the current live contribution, information, error approximation, and stopping diagnostics;
7. stop on a successful scientific rule or on a hard resource/rejection limit;
8. add the final live-point correction and return an immutable result.

Scope and central limitation. The current Phase 2 implementation does not discover a posterior from the original prior. It assumes that the input posterior samples already represent every region of material posterior mass. The fitted proposal is fixed and non-defensive: it is not mixed with the prior and is not adapted during the run. Missing proposal support can therefore bias an evidence estimate.

2 Mathematical target and change of measure

2.1 Bayesian evidence

Let $\theta \in \Theta \subseteq \mathbb{R}^d$ denote the parameters in the exact coordinate system used by the code. Let

$$L(\theta) = p(y \mid \theta) \qquad \text{and} \qquad \pi(\theta) = p(\theta)$$

be the likelihood and a *normalized* prior density with respect to the declared base measure. The evidence is

$$Z = p(y) = \int_\Theta L(\theta)\pi(\theta)\,\mathrm{d}\theta. \tag{1}$$

All logarithms in the API, implementation, history, and this document are natural logarithms.

The prior supplied to `log_prior_fn` must contain every normalization constant. For example, the normalized log density for a uniform prior on a box with side lengths $b_k - a_k$ is

$$\log \pi(\theta) = \begin{cases} -\sum_{k=1}^{d} \log(b_k - a_k), & a_k \le \theta_k \le b_k \text{ for all } k, \\ -\infty, & \text{otherwise.} \end{cases}$$

Supplying only an "inside/outside" indicator would shift $\log Z$ by the missing log-volume.

For an absolute evidence calculation, `log_likelihood_fn` must likewise represent the intended likelihood including any parameter-independent normalization terms that would matter when comparing models.

2.2 Fixed importance Morph and pseudo-likelihood

The posterior training array is used to fit one normalized importance density $q_0(\theta)$. In the standard path, q_0 is a MorphZ `GroupKDE`; a custom object satisfying the same proposal protocol may also be used.

The implementation defines

$$\Psi_0(\theta) = \frac{L(\theta)\pi(\theta)}{q_0(\theta)}, \qquad \log \Psi_0(\theta) = \log L(\theta) + \log \pi(\theta) - \log q_0(\theta). \tag{2}$$

Consequently,

$$Z = \int_\Theta \Psi_0(\theta) q_0(\theta)\, \mathrm{d}\theta. \tag{3}$$

Thus q_0 acts as the nested sampler's pseudo-prior and Ψ_0 acts as its pseudo-likelihood. This implementation fixes any tempering parameter at $\beta = 1$; there is no public `beta` argument.

The default `fixed_morph` scheme also proposes from q_0. The experimental `adaptive_morph` scheme refits a separate proposal from the current live set every 25 iterations by default, while all density evaluation and ordering continue to use q_0 and Ψ_0. It directly accepts the first proposal above the constraint without a Metropolis–Hastings correction. It therefore does not in general preserve constrained q_0, and its quadrature is heuristic and may be biased.

The code computes Equation (2) in `BatchEvaluator.evaluate` after evaluating all three log densities. It permits $-\infty$ likelihood or prior values to represent zero density. It rejects NaN and $+\infty$. If $\log L + \log \pi$ is finite but $\log q_0 = -\infty$, it raises `ProposalSupportError`; the run does not return a result.

2.3 Support and coordinate requirements

The identity in Equation (3) is useful only if

$$q_0(\theta) > 0 \quad \text{wherever} \quad L(\theta)\pi(\theta)$$

has material mass. A finite numerator divided by zero proposal density is not repairable by the current implementation.

The likelihood, prior, posterior training samples, and proposal density must also use the same:

- parameter order;
- units and transformations;
- Jacobian convention;
- dimensionality; and
- underlying density measure.

If a user changes from θ to a transformed coordinate $\phi = g(\theta)$, all model and proposal inputs must change consistently. In particular, the density in the new coordinates must include the relevant Jacobian. Merely transforming the samples without changing `log_prior_fn` and `log_likelihood_fn` violates the implemented identity.

3 Nested-importance quadrature

3.1 Nested volume under the proposal

For a pseudo-likelihood threshold λ, define the remaining proposal mass

$$X(\lambda) = \int_{\Psi(\theta)>\lambda} q(\theta)\, \mathrm{d}\theta. \tag{4}$$

Under the usual nested-sampling ordering, the evidence can be viewed as

$$Z = \int_0^1 \Psi(X)\,\mathrm{d}X.$$

With $N = \texttt{n\_live}$ live points, the code does not draw a random shrinkage factor for the quadrature. It uses the deterministic expected log-volume trajectory

$$X_i = \exp(-i/N), \qquad \log X_i = -i/N, \tag{5}$$

where $i = 1, \ldots, K$ indexes completed replacements. This choice is central: changing a stopping policy does not change Equation (5).

3.2 Dead-point contribution

At iteration i, let Ψ_i be the discarded minimum live pseudo-likelihood. The rectangular quadrature interval and contribution are

$$\Delta X_i = X_{i-1} - X_i, \tag{6}$$
$$w_i^{\mathrm{dead}} = \Delta X_i\, \Psi_i, \tag{7}$$
$$\log w_i^{\mathrm{dead}} = \log(X_{i-1} - X_i) + \log \Psi_i. \tag{8}$$

`dead_log_contribution` evaluates these quantities in log space using a stable log-difference.

3.3 Final live correction

After K completed replacements, the remaining proposal volume is $X_K = \exp(-K/N)$. The code assigns equal volume X_K/N to each current live point:

$$w_j^{\mathrm{live}} = \frac{X_K}{N} \Psi_j^{\mathrm{live}}, \tag{9}$$
$$\log w_j^{\mathrm{live}} = \log X_K - \log N + \log \Psi_j^{\mathrm{live}}, \qquad j = 1, \ldots, N. \tag{10}$$

This correction is applied on every returned result, including a hard-stop partial result.

3.4 Evidence, posterior weights, and information

Let a run over the K dead contributions followed by the N final live contributions. Finalization computes

$$\log \widehat{Z} = \mathrm{logsumexp}_a(\log w_a), \tag{11}$$
$$\log \widetilde{w}_a = \log w_a - \log \widehat{Z}, \tag{12}$$
$$\widetilde{w}_a = \exp(\log \widetilde{w}_a). \tag{13}$$

These normalized quadrature contributions are the returned posterior weights. The ordering is always all dead points first and all final live points second. The primary posterior representation is therefore

$$(\texttt{result.all\_points}, \texttt{result.posterior\_weights}).$$

The discrete information estimate is

$$\widehat{H} = \sum_a \widetilde{w}_a \left(\log \Psi_a - \log \widehat{Z}\right), \tag{14}$$

with tiny negative floating-point roundoff clipped to zero. The returned theoretical nested-sampling error approximation is

$$\texttt{logzerr} = \sqrt{\widehat{H}/N}. \tag{15}$$

It is an approximation rather than a calibration guarantee. In particular, it does not include Morph fitting uncertainty, missing modes or support, stochastic uncertainty that deterministic shrinkage omits, or invalid constrained draws.

3.5 Equal-weight resampling

`MINSResult.resample_equal` uses randomized systematic resampling. For a requested count M, it draws one $U \sim \mathrm{Uniform}(0, 1)$, forms

$$r_m = \frac{m + U}{M}, \qquad m = 0, \ldots, M - 1,$$

maps these positions through the cumulative posterior weights, and shuffles the selected indices. Repeated points are expected. This adds resampling noise, so weighted summaries should be preferred whenever the downstream tool accepts weights.

4 Exact execution path

This section follows `MINSampler.run` in execution order.

4.1 Step 0: resolve and validate configuration

`MINSConfig` is immutable and validates all settings before sampling. If neither `dlogz` nor `stopping` is supplied, the configuration is resolved to `dlogz=1e-3`. A `dlogz` value is internally represented as a one-criterion `StoppingPolicy` using `remaining_dlogz`. Supplying both interfaces is an error.

The run then creates the progress reporter, records the initial random-number generator state and start time, derives a wall-time deadline if requested, and constructs a counting `BatchEvaluator`.

4.2 Step 1: initialize the live set

The sampler requests exactly N *new* points from `proposal.sample`. It never installs the Morph training rows directly as live points. The returned array must be finite and have shape (N, d).

For each live point, the evaluator caches

$$\theta, \quad \log L, \quad \log \pi, \quad \log q, \quad \log \Psi.$$

It also draws a uniform tie breaker $u_j \in [0, 1)$ for every live point, regardless of which tie policy is selected. The initial evaluation consumes N likelihood calls and N prior calls. Initialization happens before the sampler checks run-time hard limits inside the main loop.

4.3 Step 2: check hard limits at the top of the loop

Before selecting a point for the next replacement, the sampler checks, in this order:

1. `niter >= max_iterations`;
2. the run-wide likelihood count has reached `max_likelihood_calls`;

3. the monotonic clock has reached `max_wall_time`.

The first true condition determines the hard-stop reason. These checks occur before each attempted replacement, not midway through evidence arithmetic.

4.4 Step 3: choose the worst live point

For the default `tie_policy="strict"`, the code selects

$$j_\star = \arg\min_j \log \Psi_j$$

using `numpy.argmin`. This is appropriate for ordinary continuous pseudo-likelihoods, for which exact ties have probability zero.

For `tie_policy="randomized_plateau"`, it orders the pairs

$$(\log \Psi_j, u_j)$$

lexicographically and removes the smallest pair. The auxiliary uniform value provides a continuous ordering within an exact pseudo-likelihood plateau.

4.5 Step 4: draw an independent constrained replacement

The code repeatedly draws independent batches from q. For strict ordering, a candidate is valid when

$$\log \Psi_{\text{candidate}} > \log \Psi_{j_\star}.$$

For randomized plateau ordering, it is valid when

$$\log \Psi_{\text{candidate}} > \log \Psi_{j_\star} \quad \text{or} \quad (\log \Psi_{\text{candidate}} = \log \Psi_{j_\star} \quad \text{and} \quad u_{\text{candidate}} > u_{j_\star}).$$

Valid candidates are scanned in generation order and the first is accepted. The code never selects the largest candidate in a batch. Therefore, under the stated independence and density assumptions, strict rejection targets

$$q(\theta \mid \log \Psi(\theta) > \log \Psi_{j_\star}).$$

The randomized rule targets the analogous constrained distribution on the augmented (θ, u) space.

An implementation detail relevant to cost accounting is that the entire generated batch is evaluated before the first valid row is selected. Consequently, all rows in that batch count as proposals and likelihood calls, even when an early row is accepted.

4.6 Step 5: record the dead point and replace it

After a successful constrained draw, iteration $i = K + 1$ is completed:

1. compute $\log X_i$, $\log \Delta X_i$, and the dead log contribution from Equation (8);
2. append the discarded point and all its cached density values;
3. replace its live-set row with the accepted point and its tie breaker;
4. update the cumulative dead evidence in log space;
5. increment `niter`.

Because the replacement satisfies the selected ordering rule, stored dead thresholds should be monotone.

4.7 Step 6: estimate the current live contribution

Using the post-replacement live set and X_i, the live evidence estimate is

$$\log \widehat{Z}_{\mathrm{live},i} = \log X_i + \mathrm{logsumexp}_{j=1}^{N}(\log \Psi_j^{\mathrm{live}}) - \log N. \tag{16}$$

The current total is

$$\log \widehat{Z}_{\mathrm{total},i} = \mathrm{logaddexp}\left(\log \widehat{Z}_{\mathrm{dead},i}, \log \widehat{Z}_{\mathrm{live},i}\right).$$

The code calculates the information and $\sqrt{\widehat{H}/N}$ at the same point in the iteration.

4.8 Step 7: calculate stopping diagnostics and update the streak

Every stopping metric in Section 6 is calculated after the successful replacement. Enabled criteria are evaluated using inclusive comparisons ($\leq$ or $\geq$). The configured `all`/`any` combination is gated by `min_iterations`. A passing combination increments the streak; any failure, including being below `min_iterations`, resets it to zero.

All history arrays and an optional progress callback are updated before a scientific stop is installed. Thus the final successful iteration appears in `result.history`.

4.9 Step 8: finalize on every returned path

When the loop ends with a scientific or hard-stop reason, the code:

1. sets $\log X_K = -K/N$;
2. combines all dead weights with every live weight from Equation (10);
3. recomputes final $\log Z$, H, `logzerr`, and normalized posterior weights;
4. freezes result and history arrays as read-only;
5. stores the complete configuration, random-generator states, proposal metadata representation, counts, warnings, and termination status.

The result object validates that posterior weights normalize, thresholds are monotone, and the stored evidence can be reconstructed from the stored arrays.

5 Public API and what changes for a different setup

5.1 Baseline end-to-end example

```python
import numpy as np

from mins import (
    CallableModel,
    MINSampler,
    MorphProposal,
    StoppingCriterionConfig,
    StoppingPolicy,
)

def log_likelihood(theta):
    # theta has shape (n, 2); return shape (n,)
    return -np.log(2.0 * np.pi) - 0.5 * np.sum(theta**2, axis=1)

def normalized_log_prior(theta):
```

```
    inside = np.all(np.abs(theta) <= 5.0, axis=1)
    return np.where(inside, -2.0 * np.log(10.0), -np.inf)

model = CallableModel(
    ndim=2,
    parameter_names=("x", "y"),
    log_likelihood_fn=log_likelihood,
    log_prior_fn=normalized_log_prior,
)

# Rows must use the same (x, y) coordinates as the model.
posterior_samples = np.load("posterior_samples.npy")

importance_morph = MorphProposal.fit(
    posterior_samples,
    morph_type="2_group",
    param_names=model.parameter_names,
    kde_bw="silverman",
)

sampler = MINSampler(
    model=model,
    importance_morph=importance_morph,
    proposal_scheme="fixed_morph",
    proposal_update_interval=25,
    n_live=500,
    rng=42,
    proposal_batch_size=64,
    tie_policy="strict",
)

result = sampler.run(
    dlogz=1.0e-3,
    max_iterations=20_000,
    max_proposals_per_replacement=200_000,
    max_likelihood_calls=None,
    max_wall_time=None,
    progress=True,
)

if not result.success:
    raise RuntimeError(result.termination_reason)

posterior_mean = np.average(
    result.all_points,
    axis=0,
    weights=result.posterior_weights,
)
```

5.2 Model arguments

Argument	Contract	Change it when
`ndim`	Positive integer d.	The parameter dimension changes. Then sample arrays must have d columns, `parameter_names` must have d unique entries, and `proposal.ndim` must equal d.
`parameter_names`	Unique names in column order.	The variables or their order change. The same order must be used by the posterior array and Morph grouping definitions.
`log_likelihood_fn`	Returns $(n,)$ log-likelihood values.	The data model changes. It receives (n, d) when vectorized or a single $(d,)$ row when `vectorized=False`.
`log_prior_fn`	Returns a normalized $(n,)$ log-prior.	Prior bounds, distribution, coordinates, or dimension change. Include all normalization and Jacobian terms; use $-\infty$ for zero density.
`vectorized`	Default `True`.	Set to `False` only when *both* supplied functions accept individual rows rather than batches. The adapter does not catch exceptions to guess vectorization.

For scalar user functions, the setup becomes:

```
def scalar_log_likelihood(theta_row):
    return -0.5 * theta_row[0] ** 2

def scalar_log_prior(theta_row):
    return -0.5 * theta_row[0] ** 2 - 0.5 * np.log(2.0 * np.pi)

model = CallableModel(
    ndim=1,
    parameter_names=("x",),
    log_likelihood_fn=scalar_log_likelihood,
    log_prior_fn=scalar_log_prior,
    vectorized=False,
)
```

5.3 Morph fitting arguments

Argument	Current contract/default	Effect and setup change
`posterior_samples`	Finite array (n_{train}, d), at least two rows.	Replace with representative samples for the new posterior and coordinate system. Training data are copied. There is no weights argument: weighted external samples must be converted to an appropriate unweighted training sample before this call.
`morph_type`	`None` or `"<k>_group"`, with $2 \le k \le d$.	Use, for example, `"2_group"` to compute k-order total correlations and let MorphZ greedily choose disjoint groups. The literal `"n_group"` is invalid.

Argument	Current contract/default	Effect and setup change
`group_file`	`None` or path to Morph-compatible JSON.	Use when a grouping definition was computed previously. MINS reads it and passes its contents in memory.
`groups`	`None` or in-memory Morph grouping definition.	Use for explicit groups. `groups=[]` requests independent one-dimensional KDE components.
Grouping choice	At most one of `morph_type`, `group_file`, and `groups`.	If all are omitted, the implementation also uses independent one-dimensional components. Correlated targets generally require a grouping choice that can represent those correlations.
`param_names`	Defaults to `param_0`, `param_1`, ...	Normally pass `model.parameter_names`. Names must be unique and match sample columns and group entries.
`kde_bw`	`"silverman"`; string, float, or per-parameter dictionary.	Change the KDE smoothing rule for the new training set. It is forwarded unchanged to `morphZ.GroupKDE`; it changes q, support behavior, and rejection efficiency.
`min_tc`	`None`.	Forwarded to MorphZ as the total-correlation selection threshold.
`top_k_greedy`	`1`, positive integer.	Forwarded to the greedy group selection.

The three grouping patterns are:

```
# Automatic second-order total-correlation workflow
proposal = MorphProposal.fit(
    samples,
    morph_type="2_group",
    param_names=model.parameter_names,
)

# Precomputed JSON groups
proposal = MorphProposal.fit(
    samples,
    group_file="groups.json",
    param_names=model.parameter_names,
)

# Explicit independent one-dimensional components
proposal = MorphProposal.fit(
    samples,
    groups=[],
    param_names=model.parameter_names,
)
```

`MINSampler.from_posterior_samples` is a convenience path. Every item in `morph_config` is passed to `MorphProposal.fit`, while parameter names are taken from the model; do not duplicate `param_names` inside `morph_config`:

```
sampler = MINSampler.from_posterior_samples(
    model=model,
```

```
    posterior_samples=samples,
    morph_config={
        "morph_type": "2_group",
        "kde_bw": "silverman",
    },
    n_live=500,
    rng=42,
)
```

5.4 Using a non-Morph proposal

The sampler is typed against a small normalized proposal protocol. A custom proposal must expose `ndim`, draw independent finite (n, d) samples, and return normalized $(n,)$ log densities:

```
class UniformBoxProposal:
    ndim = 2

    def sample(self, n, rng):
        return rng.uniform(-5.0, 5.0, size=(n, self.ndim))

    def log_prob(self, theta):
        inside = np.all(np.abs(theta) <= 5.0, axis=1)
        return np.where(inside, -2.0 * np.log(10.0), -np.inf)

importance_morph = UniformBoxProposal()
sampler = MINSampler(
    model=model,
    importance_morph=importance_morph,
    n_live=500,
    rng=42,
)
```

In this setup, omit every MorphZ fitting argument. The change-of-measure identity still requires the custom `log_prob` to be the normalized density of the distribution actually sampled by `sample`. Correlation among coordinates within one multivariate draw is allowed. Dependence between proposal rows or across successive draws, or state-dependent sampling, violates the current independent rejection argument.

5.5 Sampler-construction arguments

Argument	Validation/default	Practical effect
`model`	Implements the model protocol.	Defines L, normalized π, dimension, and parameter order.
`proposal`	Normalized proposal with matching `ndim`.	Defines the fixed q, initial live distribution, and constrained rejection source.
`n_live`	Required integer $N \geq 2$.	Changes the shrinkage schedule $X_i = e^{-i/N}$, initial evaluation cost, live-ESS range, resolution, theoretical error scale, and usually runtime. Larger is normally more expensive and more stable.

Argument	Validation/default	Practical effect
`rng`	Required Python integer seed or `numpy.random.Generator`.	Controls proposal resampling and tie breakers. A supplied generator is consumed in place. Recreate the sampler with the same seed and configuration for reproducibility.
`proposal_batch_size`	Default 64, positive integer.	Changes vectorized rejection throughput. A full batch is evaluated and counted. It should fit model memory. Changing it can change the random stream and realized run even with the same starting seed.
`tie_policy`	`"strict"` by default; alternatively `"randomized_plateau"`.	Use strict ordering for continuous $\log \Psi$. Use randomized plateau ordering when exact ties have nonzero probability, such as discrete or piecewise-constant targets.

5.6 Run arguments

Argument	Validation/default	When to change it
`dlogz`	`None`; if both stopping inputs are omitted, resolves to 10^{-3}. An explicit value must be positive and finite.	Use for the single `remaining_dlogz` scientific rule. It bounds the estimated change in accumulated log evidence from adding the mean-live remainder. Smaller values generally run longer.
`stopping`	`None` or a `StoppingPolicy`; mutually exclusive with `dlogz`.	Use to combine live uncertainty, remaining mass, stability, ESS, or theoretical error.
`max_iterations`	Default 10,000, positive integer.	Raise when a scientifically adequate run needs more replacements. Hitting the limit is failure, not convergence.
`max_proposals_per_replacement`	Default 100,000, positive integer.	Raise when late constrained rejection is valid but rare. Hitting it produces `constrained_sampling_exhausted`, or `plateau_stall` in the specific strict-tie case.
`max_likelihood_calls`	Default `None`; otherwise integer at least N.	Set a run-wide model-evaluation budget. The count includes the initial N live points and every evaluated proposal row.
`max_wall_time`	Default `None`; otherwise positive seconds.	Set a monotonic wall-clock budget. It is checked between batches and at the top of iterations, not by interrupting a user density function.
`progress`	`False`, `True`, or callable.	Use `True` for the optional `tqdm` display, or pass a callback for structured snapshots. It does not change quadrature.

5.7 Setup-change checklist

New setup	Arguments and contracts that must change
Different dimension or variables	Change `ndim`, `parameter_names`, both density functions, posterior sample columns, and proposal dimension. Refit the proposal. Reconsider `morph_type` because $k \leq d$.
Different prior	Change `log_prior_fn`, including normalization and any Jacobian. Ensure the posterior training data and q cover the new posterior.
Different likelihood or data	Change `log_likelihood_fn`; regenerate representative posterior training samples and refit q.
Scalar rather than batched functions	Set `vectorized=False` and make both callables accept one $(d,)$ row. Expect lower throughput.
Correlated posterior	Use an appropriate `morph_type`, `group_file`, or nonempty `groups`; do not pass more than one. Tune `kde_bw` and verify support and rejection efficiency.
Independent posterior components	Use `groups=[]`, or omit all grouping arguments (the same independent default).
Weighted posterior input	There is no Morph weights API in this adapter. Produce a representative unweighted training array externally before `MorphProposal.fit`.
Exact likelihood/pseudo-likelihood plateaus	Set `tie_policy="randomized_plateau"`. Strict ordering can exhaust its proposal budget because equal values do not satisfy $>$.
Custom normalized proposal	Implement `ndim`, `sample(n,rng)`, and `log_prob(theta)`; construct `MINSampler` directly and omit Morph fitting arguments.
More stringent default completion	Decrease `dlogz`; also raise hard budgets so they do not stop the run first. Validate by repeated runs rather than interpreting `dlogz` as an absolute evidence error.
Hybrid completion rule	Omit `dlogz` and pass a `StoppingPolicy`. Choose tolerances, `mode`, `consecutive`, `min_iterations`, and `stability_window` based on repeated benchmarks for the target family.
Expensive vectorized model	Tune `proposal_batch_size` to the model's throughput and memory; use `max_likelihood_calls` or `max_wall_time` as safeguards.
Reproducible comparison	Use explicit seeds, keep `proposal_batch_size` and all policies fixed, record `result.config`, and compare complete independent runs.

6 Current scientific stopping rules

6.1 Remaining log-evidence increment

The estimated increment from adding the mean-live remainder to the accumulated dead evidence is

$$\Delta \log Z_{\mathrm{rem}} = \log\left(\widehat{Z}_{\mathrm{dead}} + \widehat{Z}_{\mathrm{live}}\right) - \log \widehat{Z}_{\mathrm{dead}} = \log\left(1 + \frac{\widehat{Z}_{\mathrm{live}}}{\widehat{Z}_{\mathrm{dead}}}\right). \tag{17}$$

The criterion

$$\texttt{remaining\_dlogz} \leq \tau$$

passes at equality and requires a positive finite tolerance; τ need not be less than one. Before finite positive dead evidence has accumulated, the diagnostic is $+\infty$ and cannot pass.

The call

```
result = sampler.run(dlogz=1.0e-3)
```

is exactly a one-criterion `remaining_dlogz` policy with $\tau =$ `dlogz`, `mode="all"`, one required consecutive pass, and no minimum iteration gate. Omitting both `dlogz` and `stopping` selects this same 10^{-3} behavior.

6.2 Remaining live-evidence fraction

The live fraction is

$$f_{\mathrm{live}} = \frac{\widehat{Z}_{\mathrm{live}}}{\widehat{Z}_{\mathrm{total}}} = \exp\left(\log \widehat{Z}_{\mathrm{live}} - \log \widehat{Z}_{\mathrm{total}}\right). \tag{18}$$

The criterion

$$\texttt{remaining\_fraction} \leq \tau$$

passes at equality. Its tolerance must satisfy $0 < \tau < 1$. This remains an independently selectable explicit criterion.

The exact relationship between the two diagnostics is

$$\Delta \log Z_{\mathrm{rem}} = -\log(1 - f_{\mathrm{live}}). \tag{19}$$

They are similar only for a small remainder and must not be conflated.

6.3 Live effective sample size

For $a_j = \Psi_j^{\mathrm{live}}$, the code computes the Kish effective sample size in log space:

$$N_{\mathrm{eff,live}} = \frac{\left(\sum_{j=1}^{N} a_j\right)^2}{\sum_{j=1}^{N} a_j^2}. \tag{20}$$

The value is clipped to $[1, N]$. Equal finite live contributions give $N_{\mathrm{eff,live}} = N$. When every live contribution is zero ($\log \Psi_j = -\infty$), the implementation also defines the live ESS as N, the remaining fraction as zero, the live-mean RSE as zero, and the live log-evidence error as zero, provided the surrounding evidence state is numerically consistent.

The selectable criterion is

$$\texttt{live\_ess} \geq \tau,$$

with $1 \leq \tau \leq N$. It is the only currently implemented criterion that uses a greater-than-or-equal comparison.

6.4 Live-mean relative standard error

The code derives

$$\mathrm{RSE}_{\mathrm{live}} = \sqrt{\max\left(\frac{N/N_{\mathrm{eff,live}} - 1}{N - 1}, 0\right)}. \tag{21}$$

This is always recorded as `live_mean_rse`, but it is not a separately selectable criterion in the current public policy names.

6.5 Finite-live contribution to log-evidence error

The first-order delta-method diagnostic is

$$\sigma_{\log Z,\mathrm{live}} \approx f_{\mathrm{live}}\ \mathrm{RSE}_{\mathrm{live}}\,. \tag{22}$$

The criterion

$$\texttt{live\_logz\_error} \le \tau$$

requires $\tau > 0$. It estimates only uncertainty transmitted from representing the remaining integral by the finite live-set mean. It excludes proposal fitting, support, mode coverage, shrinkage, and constrained-draw validity. Equal live contributions produce zero empirical live-mean uncertainty, so this rule is unsafe on its own. The recommended policy pairs it with a `remaining_dlogz` completion guard.

6.6 Recent evidence stability

For a configured window $W = \texttt{stability\_window}$, the diagnostic is

$$S_i = \max_{r=i-W+1,\ldots,i} \log \widehat{Z}_r - \min_{r=i-W+1,\ldots,i} \log \widehat{Z}_r. \tag{23}$$

The value is NaN and the criterion cannot pass until exactly W completed estimates are available. The selectable condition

$$\texttt{logz\_stability} \le \tau$$

requires $\tau > 0$ and $W \ge 2$. Stability is supporting evidence only: a biased or mode-missing estimate can be stable.

6.7 Theoretical nested-sampling error

The condition

$$\texttt{logzerr} \le \tau$$

uses Equation (15) and requires $\tau > 0$. It inherits all limitations of the deterministic-shrinkage information approximation and is not a complete calibration guarantee.

6.8 Criterion summary

Name	Passing comparison	Tolerance validation
`remaining_fraction`	$f_{\text{live}} \le \tau$	$0 < \tau < 1$
`remaining_dlogz`	$\Delta \log Z_{\text{rem}} \le \tau$	$\tau > 0$
`live_logz_error`	$\sigma_{\log Z,\text{live}} \le \tau$	$\tau > 0$
`logz_stability`	$S_i \le \tau$	$\tau > 0$
`live_ess`	$N_{\text{eff,live}} \ge \tau$	$1 \le \tau \le N$
`logzerr`	$\sqrt{\widehat{H}/N} \le \tau$	$\tau > 0$

Criterion names must be unique within one policy. All tolerances must be finite real numbers and Boolean values are rejected.

6.9 Combining, gating, and persisting criteria

For enabled Boolean criterion results $c_1, \ldots, c_m$:

$$c_{\text{criteria}} = \begin{cases} \bigwedge_{r=1}^{m} c_r, & \texttt{mode="all"}, \\ \bigvee_{r=1}^{m} c_r, & \texttt{mode="any"}. \end{cases}$$

The combined condition at completed iteration i is

$$c_i = [i \ge \texttt{min\_iterations}] \wedge c_{\text{criteria}}.$$

The run-local streak is updated exactly as

$$s_i = \begin{cases} s_{i-1} + 1, & c_i \text{ is true}, \\ 0, & c_i \text{ is false}. \end{cases}$$

Scientific stopping occurs when

$$s_i \ge \texttt{consecutive}.$$

`consecutive` must be a positive integer, `min_iterations` must be an integer at least zero, and `stability_window` must be an integer at least two. Even when `min_iterations=0`, the earliest scientific stop is after the first completed replacement because policies are not evaluated on the initial live set.

A current experimental hybrid example is:

```
n_live = sampler.n_live
stopping = StoppingPolicy(
    criteria=(
        StoppingCriterionConfig("remaining_dlogz", 1.0e-2),
        StoppingCriterionConfig("live_logz_error", 2.0e-3),
        StoppingCriterionConfig("logz_stability", 5.0e-3),
    ),
    mode="all",
    consecutive=3,
    min_iterations=n_live,
    stability_window=n_live,
)

result = sampler.run(
```

```
    stopping=stopping,       # do not also pass dlogz
    max_iterations=20_000,
)
```

These threshold values are illustrative calibration choices, not universal constants. They require repeated-run calibration using independent seeds, known benchmarks where possible, proposal diagnostics, and a live count appropriate to the new target.

7 Exact termination semantics

7.1 Successful scientific termination

There are exactly two termination reasons for which the result sets `success=True`:

`termination_reason`	**Exact meaning**
`remaining_evidence`	The run used the default/legacy-compatible `dlogz` interface, and after a completed replacement its remaining-log-evidence-increment policy reached the required streak.
`stopping_criteria`	The run used an explicit `StoppingPolicy`, and after a completed replacement its combined, gated, consecutive rule passed.

Scientific stopping is checked only after a replacement is completed and recorded. There is no scientific stopping check on the initial live set before the first replacement.

7.2 Returned hard-stop failures

The following reasons return an immutable, internally consistent `MINSResult` with the current final-live correction, but set `success=False`:

`termination_reason`	**Trigger**
`max_iterations`	At the top of an iteration, completed replacements satisfy `niter >= max_iterations`.
`max_likelihood_calls`	The run-wide count is exhausted either at the top of the loop or before a new constrained batch. The next batch size is capped by the remaining budget, so the configured count is not intentionally exceeded.
`max_wall_time`	The monotonic deadline is reached at the top of the loop or between constrained proposal batches. A running user function is not interrupted.
`constrained_sampling_exhausted`	One replacement consumes `max_proposals_per_replacement` evaluated proposal rows without a valid constrained candidate.
`plateau_stall`	The constrained proposal limit is exhausted under strict ordering and more than one current live point equals the minimum threshold. This is a diagnostic relabeling of that exhausted attempt, indicating an exact-tie plateau.

A hard-stop evidence value is a *partial-run estimate*. It includes all completed dead contributions and the current mean-live remainder, but a hard limit is not evidence of convergence. Code consuming results should always check both fields:

```
if not result.success:
    print("MINS did not converge scientifically:", result.termination_reason)
```

7.3 Ordering and simultaneous conditions

At the top of a new loop pass, hard checks have the priority order `max_iterations`, then `max_likelihood_calls`, then `max_wall_time`. Inside constrained rejection, wall time is checked before the remaining likelihood budget for each new batch.

After a successful replacement, the scientific policy is evaluated immediately. If it passes, its scientific reason is installed before the next top-of-loop hard checks. For example, if an iteration both reaches `max_iterations` and satisfies the scientific rule, the scientific termination wins because the iteration-limit check would occur only on the next pass.

7.4 Raised errors: no result is returned

Some failures mean that no statistically coherent result can be formed. They raise an exception rather than returning `success=False`:

Exception class/category	Examples
`ConfigurationError`	Invalid live count, limits, tie policy, stopping name/tolerance, duplicate criteria, or simultaneous `dlogz` and `stopping`.
`InvalidModelOutput`	Wrong output shape, NaN, $+\infty$, nonfinite input point, or invalid computed $\log \Psi$.
`InvalidProposalOutput`	Wrong sample/density shape, nonfinite samples, NaN, or $+\infty$ proposal log density.
`ProposalSupportError`	Finite $\log L + \log \pi$ with $\log q = -\infty$.
`NumericalInvariantError`	An impossible evidence/metric state, nonfinite final evidence, materially negative information, or posterior weights that do not normalize.
`MissingOptionalDependency`	MorphZ is unavailable when fitting a Morph proposal, or `tqdm` is unavailable when `progress=True`.
User exception	An exception raised by a supplied likelihood, prior, proposal, progress callback, or backend is not guessed away by the adapter.

8 Result interpretation and diagnostics

8.1 Core scalars and status

- `logz`, `information`, and `logzerr` contain the final deterministic-shrinkage quadrature summary.
- `success` and `termination_reason` must be read before treating the result as scientifically complete.

- `niter`, `nlive`, `n_likelihood_calls`, `n_prior_calls`, and `n_proposals` describe cost.
- `config` stores the fully resolved immutable run configuration, including the resolved stopping policy.
- initial and final random-generator states and proposal metadata are retained as representations for auditability.

8.2 Dead and live arrays

The result separately stores points, $\log L$, $\log \pi$, $\log q_0$, $\log \Psi_0$, and tie breakers for dead and final-live populations. Dead points additionally store $\log X_i$ and their log quadrature contributions. All arrays are copied, read-only, and validated.

`result.log_posterior_weights` has length $K + N$ and follows the dead-then-live ordering. Convenience properties use the same order:

```
points = result.all_points
log_psi0 = result.all_log_psi0
weights = result.posterior_weights
```

8.3 Per-iteration history

Every history field has length `niter`. The history includes:

- discarded threshold, $\log X_i$, and $\log \Delta X_i$;
- dead, live, and total $\log Z$;
- information and theoretical `logzerr`;
- remaining fraction, remaining log-evidence increment, live ESS, live-mean RSE, live log-evidence error, stability, and stopping streak;
- minimum, median, and maximum live $\log \Psi$;
- proposal rows used for each completed replacement, cumulative likelihood calls, acceptance fraction, and elapsed seconds.

If no replacement completes, all history arrays are empty. Diagnostic summary helpers report NaN for unavailable final floating-point metrics and zero for the final streak.

8.4 Progress semantics

`progress=True` displays an indeterminate live iteration count rather than a percentage of the hard iteration limit. It always shows live count, likelihood calls, efficiency, current evidence, theoretical error, and stopping streak. Criterion-specific diagnostics appear only when enabled, proposal status appears only after adaptive updates, and remaining fraction is omitted from the terminal display. A callable still receives the complete mapping after every completed iteration, including a `criterion_<name>_met` integer flag for every enabled criterion.

The acceptance fraction stored in history is

$$\frac{\text{completed replacements}}{\text{evaluated proposal rows}},$$

where the denominator includes constrained-replacement proposal rows but excludes the initial N live draws. It is not the fraction of Python batch calls and not the number of mathematically valid rows divided by proposals.

9 Practical tuning and validation

9.1 Choosing the live count

Increasing N makes the volume path in Equation (5) finer, increases the cost of initialization and every live-set diagnostic, raises the maximum possible live ESS, and reduces the formal $\sqrt{\widehat{H}/N}$ scale for fixed H. It does not cure missing proposal support. Choose N by repeated-run calibration on targets similar to the intended analysis.

9.2 Choosing a proposal batch size

A larger batch can improve vectorized likelihood throughput, but every row is evaluated and counted even if the first row is valid. Very large batches can therefore waste expensive evaluations when acceptance is high. Very small batches can create Python/backend overhead. The batch size does not change the formal constraint or quadrature, but it can change the realized random stream because unused rows from an accepted batch are not retained for the next iteration.

9.3 Recognizing rejection difficulty

As the threshold rises, the probability under q of satisfying the constraint may become small. Warning signs are:

- falling `history.acceptance_fraction`;
- large `history.proposals`;
- rapidly increasing likelihood calls per iteration;
- `constrained_sampling_exhausted`; or
- `plateau_stall` under strict ties.

Raising `max_proposals_per_replacement` only gives valid rejection more time; it does not improve q. Proposal structure, bandwidth, posterior coverage, parameterization, and plateau policy should be investigated.

9.4 Recommended validation for a new setup

1. Verify the normalized prior analytically or numerically.
2. Confirm the posterior sample column order and units against `parameter_names`.
3. Inspect Morph metadata for selected groups and single parameters.
4. Test `proposal.sample` and `proposal.log_prob` together in tail and boundary regions.
5. Run multiple independent seeds and compare $\log Z$, cost, termination reasons, and posterior summaries. To assess Morph-fitting variation as well as sampler variation, repeat the training/refitting stage rather than changing only the run seed.
6. Where possible, compare with an analytic target, direct importance sampling under the same fixed q, or another evidence estimator.
7. Benchmark stopping policies on the target family rather than assuming the example hybrid thresholds are universal.
8. Retain weighted samples and inspect posterior ESS and weight concentration before equal-weight resampling.

10 Implementation map

Source file	Responsibility
`src/mins/model.py`	Model protocol, callable adaptation, point-shape validation.
`src/mins/proposals/base.py`	Normalized proposal sampling/density protocol.
`src/mins/proposals/morph.py`	MorphZ fitting, grouping inputs, fixed `GroupKDE` sampling and row-wise normalized log-density adapter.
`src/mins/constrained.py`	Batch density evaluation, count tracking, support checks, and independent strict/lexicographic constrained rejection.
`src/mins/quadrature.py`	Log-space deterministic volumes, dead/live contributions, evidence, information, posterior weights, and theoretical error.
`src/mins/stopping.py`	Stopping configuration, finite-live diagnostics, criterion comparisons, combination, iteration gate, and consecutive streak.
`src/mins/config.py`	Immutable run configuration, legacy stopping resolution, and validation.
`src/mins/sampler.py`	Serial state machine, hard-limit ordering, history/progress snapshots, termination reason, finalization, and result construction.
`src/mins/results.py`	Immutable result/history containers, consistency checks, convenience arrays, and systematic equal-weight resampling.
`src/mins/diagnostics.py`	Read-only summary diagnostics computed from a completed result.
`src/mins/progress.py`	Silent, callback, and optional `tqdm` progress reporters.

11 Concise user decision guide

1. **Do the training samples cover all posterior modes?** If not, do not trust Phase 2 evidence; regenerate the posterior input.
2. **Are prior and proposal densities normalized in the exact sample coordinates?** If not, fix them before running.
3. **Is the transformed pseudo-likelihood continuous?** Use `tie_policy="strict"`. If exact plateaus have nonzero probability, use `"randomized_plateau"`.
4. **Is the posterior correlated?** Select a suitable Morph grouping workflow rather than relying automatically on independent marginals.
5. **Which scientific stop is intended?** Use only `dlogz` for the remaining log-evidence increment, or only `stopping` for an explicit policy.
6. **Did the run stop successfully?** Require `result.success` before treating it as scientifically converged, and inspect the exact reason and final diagnostics.
7. **Does the downstream tool accept weights?** Use the native weighted representation if possible; resample only when necessary.

12 Summary of guarantees and non-guarantees

Under the stated contracts, the current code provides:

- a fixed normalized proposal used consistently for sampling and density evaluation;
- a clear change-of-measure identity;
- independent generation-order constrained rejection;
- deterministic log-volume quadrature with dead and live corrections;
- normalized posterior weights and reconstructible evidence;
- reproducible explicit RNG ownership;
- exact policy comparison, persistence, and termination status; and
- valid partial quadrature output for defined hard stops.

It does not guarantee:

- posterior discovery or recovery of modes absent from training data;
- defensive proposal support;
- adaptation of q during the run;
- a complete uncertainty estimate in `logzerr`, `live_logz_error`, or stability;
- efficient rejection at every threshold;
- that example stopping tolerances are calibrated for a new problem; or
- scientific convergence when a resource limit is reached.

The minimum safe interpretation rule is: check `result.success`, verify the exact `termination_reason`, inspect the final stopping and weight diagnostics, and validate the entire setup over repeated independent runs.